

Edgar H. Sibley
Panel Editor

The elimination of ambiguity, inconsistency, and incompleteness in a Software Requirements Specification (SRS) document is inherently difficult, due to the use of natural language. The focus here is a survey of available techniques designed to reduce these negatives in the documentation of a software product's external behavior.

A COMPARISON OF TECHNIQUES FOR THE SPECIFICATION OF EXTERNAL SYSTEM BEHAVIOR

ALAN M. DAVIS

During the requirements specification phase of the software development life cycle, it is necessary to describe in detail the expected external behavior of the system to be built. This behavior is recorded in a document commonly called the Software Requirements Specification (SRS). Currently, most SRSs are written in natural language. Unfortunately, natural language is inherently ambiguous, resulting in documents that are ambiguous, inconsistent, and incomplete. With software being used in more and more life-critical applications, it becomes increasingly important to find ways of reducing the ambiguity, inconsistency, and incompleteness in SRSs.

Software Engineering is the application of scientific principles to: (1) the orderly transformation of a problem into a working software solution, and (2) the subsequent maintenance of that software through the end of its useful life. People and projects that follow an engineered approach to software development generally pass through a series of phases. Royce [48] was the first to coin the phrase *waterfall model* to characterize the

series of software engineering phases. Figure 1 shows the original model that Royce presented. This model, however, fails to show the symmetry that exists between the earlier and later phases of the development life cycle. For this reason, the symmetrical view of the development process, as shown in Figure 2, is generally more useful. In this model, the path indicated by a series of bold arrows is the primary path, consisting of the following steps:

- *Software Requirements.* The activity that includes analysis of the software problem at hand and concludes with a complete specification of the expected external behavior of the software system to be built.
- *Preliminary Design.* The activity that decomposes the software system into its actual constituent (architectural) components and then iteratively decomposes these components into successively smaller subcomponents.
- *Detailed Design.* The activity of defining and documenting algorithms for each component that will be realized as code.
- *Coding.* The activity of transforming the algorithms defined during the detailed design stage into a computer-understandable language.
- *Unit Testing.* The activity of checking each coded module for the presence of bugs.

Some of the material in this paper has been excerpted from a forthcoming book by the author, entitled *Software Requirements: Analysis and Specification*, Prentice-Hall, Englewood Cliffs, New Jersey.

© 1988 ACM 0001-0782/88/0900-1098 \$1.50

- **Integration Testing.** The activity that interconnects sets of previously tested modules to ensure that the sets behave as well as their independently tested module components did.
- **System Testing.** The activity of checking that the entire software system (i.e., fully integrated) embedded in its actual hardware environment behaves according to the SRS.
- **Delivery, Production, Deployment.** After final system testing, the software and its surrounding hardware become operational.
- **Maintenance and Enhancement.** Maintenance is the continued detection and repair of bugs after deployment, and enhancement is the addition of new capabilities.

The remaining three boxes in Figure 2 are present as it is best to figure out how you are going to test before the testing stage. Starting from the top and working down:

- **Software System Test Planning.** The activity whose primary purpose is to assess how the software system will be tested for conformity to the software requirements. Another purpose of software system test plan-

ning is the thorough examination of the SRS to determine whether it is verifiable, i.e., whether or not it is written in a manner that makes it possible to verify that the software meets the requirements.

- **Integration Test Planning.** The activity whose purpose is to generate and document plans and procedures to effect an orderly system integration.
- **Unit Test Planning.** The activity whose purpose is to generate and document plans and procedures to thoroughly test each module independently.

As determined in Figure 1, the requirements phase is of extreme importance. During this phase, an SRS is written whose purpose is to:

- contain a complete description of *what* the software will do without describing *how* it will do it (i.e., it will contain a complete description of the software product's external behavior but not any information concerning the software product's internal structure).
- serve as *the* basis for all design activities; and
- serve as *the* basis for all system test planning.

Various terms are used to denote the writing of an SRS [25]: requirements [2, 9, 32, 49], specification [16], system design [47], functional specification [8], external design [56], software requirements specification [37, 45], and input/output perspective [39].

Errors in SRSs are common [5, 14], often not detected until after product delivery [10], and cost far more to fix the later they are detected [9, 18, 30]. Therefore, it is of extreme importance to find ways to reduce the most common types of errors in SRSs: incorrect facts, omissions, inconsistencies, and ambiguities [5]. The use of formal models to specify a product's external behavior reduces the number of errors that may remain latent in the SRS because the associated techniques and tools can alert the SRS writers to many potential errors.

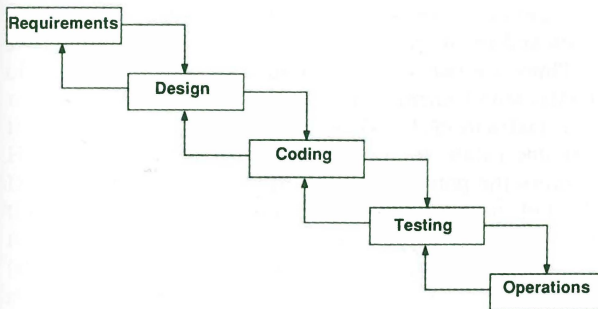


FIGURE 1. Standard Waterfall Life Cycle Model

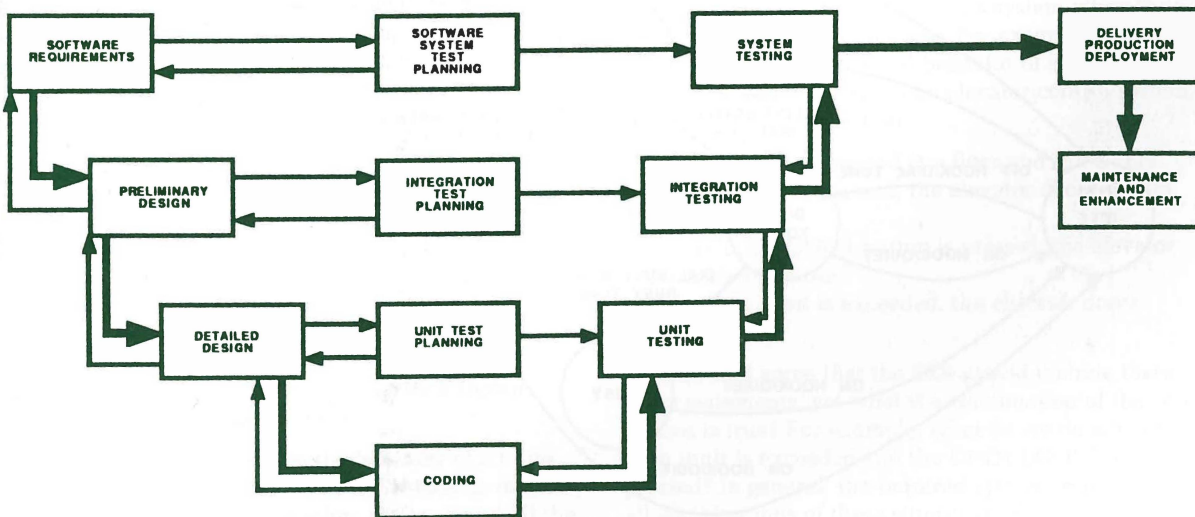


FIGURE 2. A Software Engineering Life Cycle

A SURVEY OF TECHNIQUES

Maintaining One's Perspective

Most requirements specifications today are written in natural language and range in length from a few pages to 5,000 pages or more. The size of the document rarely has any relationship to the complexity of the problem. The larger documents are usually created by organizations that are attempting more "completeness." In this attempt to create a comprehensive document, they often create monsters. The larger the natural language document becomes, the more impossible consistency and maintenance become. The fear of ruining a document becomes a major deterrent to the evolution of the product! While I am not opposed to large SRSs, I am opposed to large SRSs written in natural language. This is analogous to building a 100-story skyscraper. I am not opposed to 100-story office buildings, just to those constructed entirely of wood.

The solution to the skyscraper problem is simple: use steel. The solution to the SRS problem is also simple: use a formal technique. Nevertheless, wood does have its proper place, even in a 100-story building. And natural language does have its proper place in an SRS. Where do you apply a formal technique and where do you apply something else? The answer is: *use a formal technique when you cannot afford to have the requirement misunderstood.*

Criteria for Evaluating a Technique

The following subsections discuss a variety of behavioral requirements specification techniques and their underlying models. So they can be fairly judged, here are the traits that we would like to see a behaviorable specification technique exhibit:

- (1) When the technique is properly used, the resulting SRS should be helpful and understandable to non-computer-oriented customers and users.
- (2) When the technique is properly used, the resulting SRS should be able to serve effectively as the basis for design and testing.
- (3) The technique should provide automated checks for ambiguity, incompleteness, and inconsistency.
- (4) The technique should encourage the requirements writer to think and write in terms of external product behavior, not internal product components.
- (5) The technique should help organize the information in the SRS.
- (6) The technique should provide a basis for automated prototype generation.
- (7) The technique should provide a basis for automated system test generation.
- (8) The technique should be suitable to the particular application.

Finite State Machines

A *finite state machine* (FSM) is a hypothetical machine that can be in only one of a given number of states at any specific time. In response to an input, the machine generates an output and changes state. Both the output and the new state are purely functions of the current state and the input.

There are two notations commonly used to define FSMs: state transition diagrams (STD) and state transition matrices (STM). When STDs are used, a circle denotes a state, a directed arc connecting two states denotes the potential to transition between the two indicated states, and the label on the arc (which has two parts separated by a slash) denotes the input that triggers the transition and the output with which the sys-

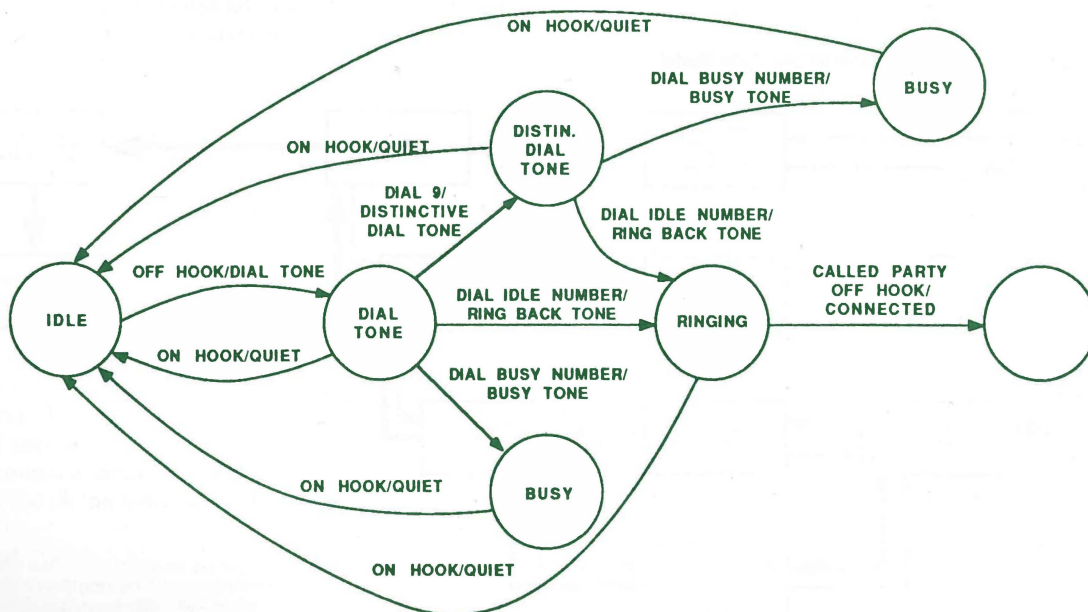


FIGURE 3. State Transition Diagram Example

STATE \ STIMULUS	LIFT PHONE	DIAL 9	DIAL 1-8	DIAL 0	HANG UP PHONE	SEVENTH DIGIT COLLECTED	OUTPUTS
IDLE	DIAL TONE STATE	—	—	—	—	—	QUIET
DIAL TONE STATE	—	DISTINCTIVE DIAL TONE	COLLECTING 7 DIGITS	OPERATOR	IDLE	—	DIAL TONE TO CALLING PARTY
DISTINCTIVE DIAL TONE STATE	—	COLLECTING 7 DIGITS	COLLECTING 7 DIGITS	OPERATOR	IDLE	—	DISTINCTIVE DIAL TONE TO CALLING PARTY
OPERATOR STATE	—	—	—	—	IDLE	—	CONNECT CALLING PARTY TO OPERATOR
COLLECTING 7 DIGITS STATE	—	COLLECTING 7 DIGITS	COLLECTING 7 DIGITS	COLLECTING 7 DIGITS	IDLE	IF CALLED PARTY BUSY, BUSY STATE; OTHERWISE, RINGING STATE	NO TONE TO CALLING PARTY
RINGING STATE	—	—	—	—	—	—	RINGING SIGNAL TO CALLED PARTY; RINGBACK TONE TO CALLING PARTY
BUSY STATE	—	—	—	—	—	—	BUSY TONE TO CALLING PARTY

FIGURE 4. Finite State Machine Telephony Example

tem responds. For example, Figure 3 shows a small fragment of the specification of the external behavior of a telephone switching system. Starting with the IDLE state on the left, there is exactly one arc emanating out of it. It is labeled "OFF HOOK/DIAL TONE" and terminates at the DIAL TONE state. That means that when the system is in IDLE and receives the stimulus OFF HOOK, it will generate a DIAL TONE and enter the DIAL TONE state. There are four arcs emanating from the DIAL TONE state, and the path taken will be a function of whether the system receives an ON HOOK (system will generate QUIET and return to the IDLE state), a "9" (system will generate a DISTINCTIVE DIAL TONE and enter the DISTINCTIVE DIAL TONE state), a DIAL IDLE NUMBER (system will generate a RING BACK TONE and enter the RINGING state), or a DIAL BUSY NUMBER (system will generate a BUSY TONE and enter the BUSY state). It is possible to describe the behavior shown in Figure 3 in natural language. We just have. As soon as you stop talking however, in terms of states, inputs, and outputs, the behavioral description becomes extremely hard to create or understand without ambiguity.

Remember that Figure 3 is part of an SRS; it is not a design. The implication to the designer reading Figure 3 in the SRS is:

Dear Designer: I don't care how you design the system, but when it is all done and I observe it as a black box, I want it to behave externally as if it were designed as the finite state machine shown. Signed, the Requirements Writer.

The second way to describe the behavior of a finite state machine is the STM. In an STM, a table is drawn with all the possible states labeling the rows and all the possible stimuli labeling the columns. In the Mealy model, the next state and the required system response

appear in each intersection. In the Moore model, the intersections of columns and rows contain only the name of the next state, and a separate column is drawn to indicate the system response associated with each state. The choice between Mealy and Moore should be guided by which one is most expressive and understandable for the particular application. Figure 4 shows a small subset of a telephone switching system's external behavior using a Moore STM.

Finite state machines have been used effectively for requirements specifications for telephony applications [38, 58]. They also serve as the underlying model of many of the techniques that follow.

Decision Tables and Decision Trees

Sometimes there is a need to describe the required external behavior of some aspect of a system when the FSM approach makes no sense. For example, if we want to specify the external behavior of a software-controlled elevator door on an elevator control system, we would easily agree that:

- If the elevator is stopped at a floor and the OPEN DOOR button is pressed, the elevator doors should open.
- If the CLOSE DOOR button is pressed, the elevator doors should close.
- If the time limit is exceeded, the elevator doors should close.

We would agree that the SRS should include these three statements, yet what if a combination of the conditions is true? For example, what do we do when the time limit is exceeded and the OPEN DOOR button is pressed? In general, the required system responses to all combinations of these stimuli are difficult to describe using a finite state machine, and even more difficult to describe in English. One simple solution is the

One column for every combination of condition outcomes

	RULE #1	RULE #2	RULE #3	RULE #4	RULE #5	RULE #6	RULE #7	RULE #8
CONDITION #1	Y	Y	Y	Y	N	N	N	N
CONDITION #2	Y	Y	N	N	Y	Y	N	N
CONDITION #3	Y	N	Y	N	Y	N	Y	N
ACTION #1	X	X	X				X	
ACTION #2		X	X	X				
ACTION #3						X		
ACTION #4	X			X	X			

List all conditions that influence decisions

All combinations of answers

List all the possible decisions

Fill in with the actual required system behavior

FIGURE 5. Decision Table

decision table. Decision tables and decision trees have been known for decades. Their uses and capabilities were recently explored thoroughly by Chvalovsky [17] and Moret [41].

To construct a *decision table* (see Figure 5), first draw a row for each condition (or stimulus) that will be used in the process of making a decision. Next draw a column for every possible combination of outcomes of those conditions (if there are n conditions and each has a binary result, then there will be 2^n columns). Finally, add rows at the bottom of the table for each action (or response) that you may want the system to perform (or generate), and fill in the boxes to reflect which actions you want performed for each combination of conditions. Figure 6 shows a decision table for a software-controlled elevator door problem. Although it has

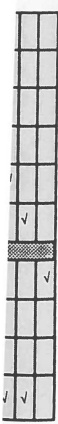
64 columns, it is easier to create and understand than an STD or STM and is considerably easier to understand than the corresponding English!

A *decision tree* captures the same information as a decision table but is graphical rather than tabular. Basically it is a flow chart without loops and without fan-in (i.e., two arrows pointing to the same node). The decision tree often takes up more room than the corresponding decision table because of the blank space required between the nodes, but because the decision tree captures the order of evaluating the conditions, it is possible to save considerable space on some problems due to the all-encompassing effect of some conditions. For example, the decision tree in Figure 7 shows the same behavior as the decision table in Figure 6.

Decision tables can easily be “automated” with any

[illegible]

FIGURE 6. A Decision Table for an Elevator Door Control



Program Design Language

Many people who see PDL in SRSs claim that the requirements writers have overstepped their bounds and fallen into design. One reason for this may be that these people are voicing an objection because they are so familiar with seeing PDL during design that they may not even read the SRS PDL to find out what it says. It is not the format that makes something appropriate for the requirements or the design phase; it is the content.

phase is the incredible ease with which one can fall into design. The best advice to help one stay out of design is to initially select all the nouns, verbs, and adjectives to be used in the SRS to represent things that can be seen, smelled, felt, heard, or tasted by somebody who is an external observer of the final system. This approach of preselection was demonstrated for telephony by Casey [13] and Davis [27] and for a flight control application by Heninger [36].

FIGURE 8. A PDL for an Elevator Door Control

Structured Analysis/Real-Time

Two extensions of Structured Analysis (SA) [29] were recently proposed by Hatley [35] and Ward [54]. These extensions have been termed *Structured Analysis/Real-Time* (SA/RT). Hatley's extensions add control diagrams and control specs to their data counterparts. For example, in addition to the standard Data Flow Diagram (DFD), he adds the Control Flow Diagram (CFD). In addition to the process spec (DeMarco, et al., also called this the "mini-spec" [29]), he adds the control spec. Since Hatley appreciates the unique difference between requirements and design, he defines the requirements dictionary as the repository for all the processes, data, and controls present in the set of diagrams. The CFD looks just like a DFD but all arrows are drawn dotted, and the arrows denote control rather than data. Control implies either a command or a report of an occurrence of something happening. DFDs and CFDs are usually at the same level of abstraction. Thus, CFDs are associated with a corresponding DFD and are *slaves* (in the electrical engineering sense) of their DFD. Control specs are Mealy or Moore model state transition diagrams or matrices for finite state machines corresponding to a CFD. Hatley has applied his extensions to a commercial flight management computer system.

Ward's extensions are very similar to Hatley's. Rather than two types of diagrams, the DFD and the CFD, Ward allows the introduction of control information in the standard DFD. Thus, in addition to the usual processes (i.e., data transformations), data flows, and data stores (i.e., data bases) in a DFD, he allows for control transformations, event flows, and buffers. Furthermore, Ward differentiates between two types of data flows:

- (1) *Discrete Data*. A single item of data (e.g., current floor in an elevator control example); and
- (2) *Continuous Data*. A source of constantly available and perhaps continuously changing data (e.g., current position of doors where 0 is open and 1 is closed and all values in between are possible);

and three types of event flows:

- (1) *Signal*. Reporting of an event (e.g., doors have reached their full open position);
- (2) *Activation*. A direct overt action to initiate control process (e.g., the elevator control system activating the elevator door control process upon arrival at a floor); and
- (3) *Deactivation*. A direct overt action to stop another process (e.g., the elevator control system deactivating the usual elevator dispatching process upon detecting a fire in the building).

In addition, Ward has given the SRS writer the ability to specify a Mealy model finite state machine description for each control transformer in the DFD. Notationally, rectangles are used for the states in the STD to prevent accidentally mistaking it for a DFD, as shown in Figure 9 [54]. Ward's final extension provides for the hierarchical definition of transformation schemas, that is, hierarchical finite state machines. His brief description falls far short of the elegance and rigor of statecharts, which we will discuss next.

Statecharts

Statecharts, extensions to Finite State Machines (FSM), were recently proposed by Harel [33]. Statecharts make it even easier to model complex real-time system behavior without ambiguity. The extensions provide a notation and set of conventions that facilitate the hierarchical decomposition of FSMs and a mechanism for communication between concurrent FSMs. The first extension to FSMs suggested by Harel allows a transition that is a function not only of an external stimulus but also of the truth of a particular condition. As shown in Figure 10, the condition becomes a convenient notation in telephony, for example, to specify that the new state (BUSY TONE or RING BACK TONE) is a function of whether or not the party being called (i.e., the CALLEE) is busy.

The next simple extension is the *superstate*. The superstate can be used to aggregate sets of states with common transitions. For example, let us assume that we have a situation in which two states, S_1 and S_2 , both transition to state S_3 upon the same stimulus, i . Using regular STDs, Figure 11(a) would result. With the superstate concept, however, we can use the shorthand notation shown in Figure 11(b). Telephony, once again, provides an excellent example. In particular, the caller going on-hook causes a transition from most call states back to the idle state. Figure 12 demonstrates this application of the superstate. It is apparent that superstates can be conceptualized and defined before they are decomposed into subordinate states; thus superstates pro-

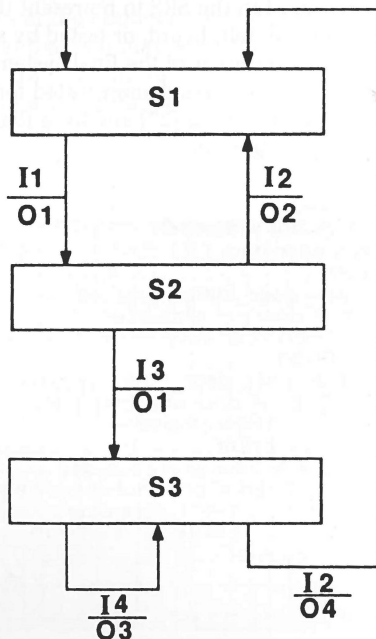


FIGURE 9. Ward-style State Transition Diagram

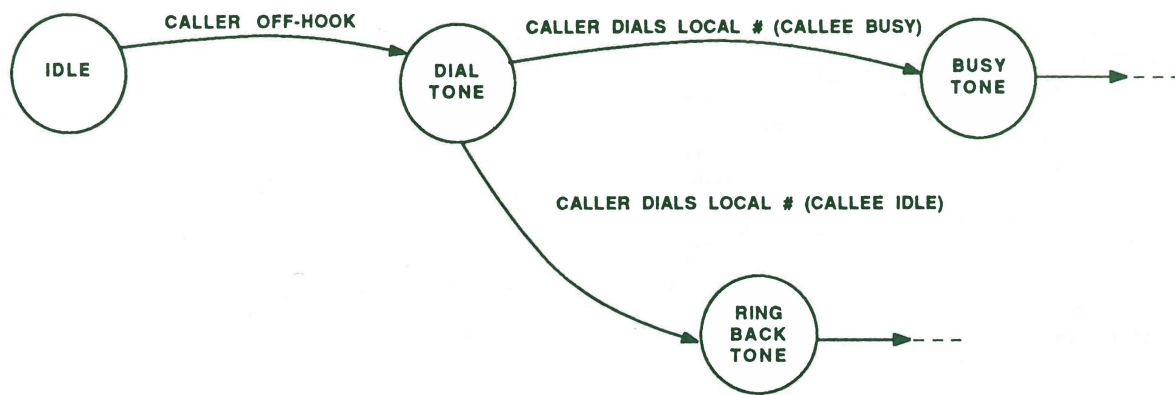


FIGURE 10. Using the Condition Transition Extension for a Local Telephone Call

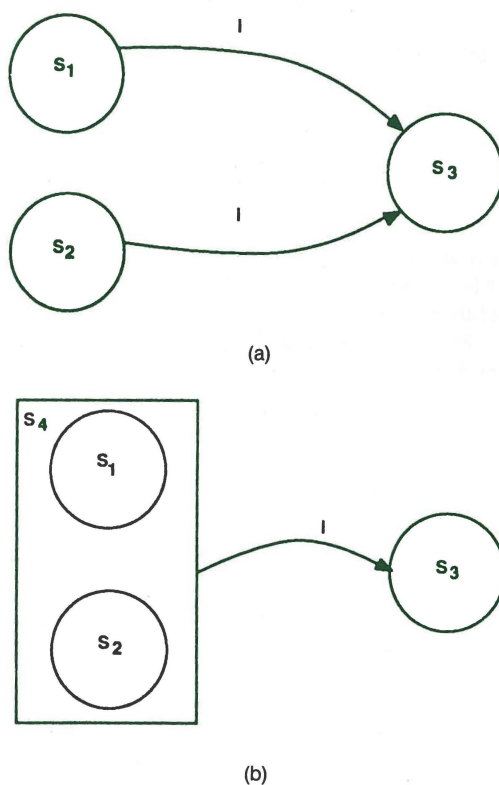


FIGURE 11. The Superstate Extension to FSMs

vide us with a formal basis for iterative refinement and successive decomposition of FSMs.

Harel also introduces the concept of the default entry state. This is the subordinate state of a superstate into which the FSM enters if no other subordinate state is specified as the next state. One specifies such a default entry state with a small arrow as shown in Figure 13. Thus, if state S_{21} were defined as the default entry state of state S_2 , Figure 13 would imply that we move transitionally from state S_1 to state S_{21} in the event of stimulus, i .

Earlier, we described the process of decomposing a state (e.g., S_2) into subordinate states (e.g., S_{21} and S_{22}). The implied semantics of this refinement are that of an *or* function; namely, that when we are in state S_2 at the higher level of abstraction, we are really in either state S_{21} or state S_{22} at the lower level of abstraction. Harel also introduces a different type of state refinement: the *and* function. The *and* decomposition in statecharts is represented by splitting a box using dashed lines. For example, Figure 14(a) shows a simple STD. Figure 14(b) shows the refinement of state S_2 into its two subordinate states, S_{21} and S_{22} . The semantics behind this notation are well defined. In particular, when we are in state S_2 , we are in states S_{21} and S_{22} . This becomes more meaningful when states S_{21} and S_{22} are further refined as shown in Figure 14(c). On receipt of stimulus i_1 , in state S_1 , we enter both states S_{211} and S_{222} simultaneously. If stimulus i_4 now arrives, we transition from states S_{211} and S_{222} simultaneously into states S_{212} and S_{223} . On the other hand, if stimulus i_6 arrives while we are in states S_{211} and S_{222} , only the S_{22} machine changes state from state S_{222} to S_{221} , leaving us in both S_{211} and S_{221} . Harel uses the term *orthogonal* to describe this type of decomposition and to emphasize the independence of state machines S_{21} and S_{22} . The simplicity, applicability, and elegance of Harel's orthogonal decomposition becomes most apparent when one compares equivalent behavioral descriptions using conventional state transition diagrams. Figure 15 shows the equivalent to Figure 14(c) using conventional STDs. Not only has the number of states increased (which makes STDs tenuous at best for describing complex system behavior), but also the monolithic structure of Figure 15 makes it extremely hard to understand in comparison to the statechart of Figure 14(c).

Earlier, we discussed the ability to specify the triggering of a transition not only on the arrival of a stimulus, but also on the truth of a condition. In concert with the orthogonality refinement, Harel has also defined the ability to specify a transition based on whether an FSM is in a particular state. For example, Figure 16 shows a modification of Figure 14(c) in order to specify that the

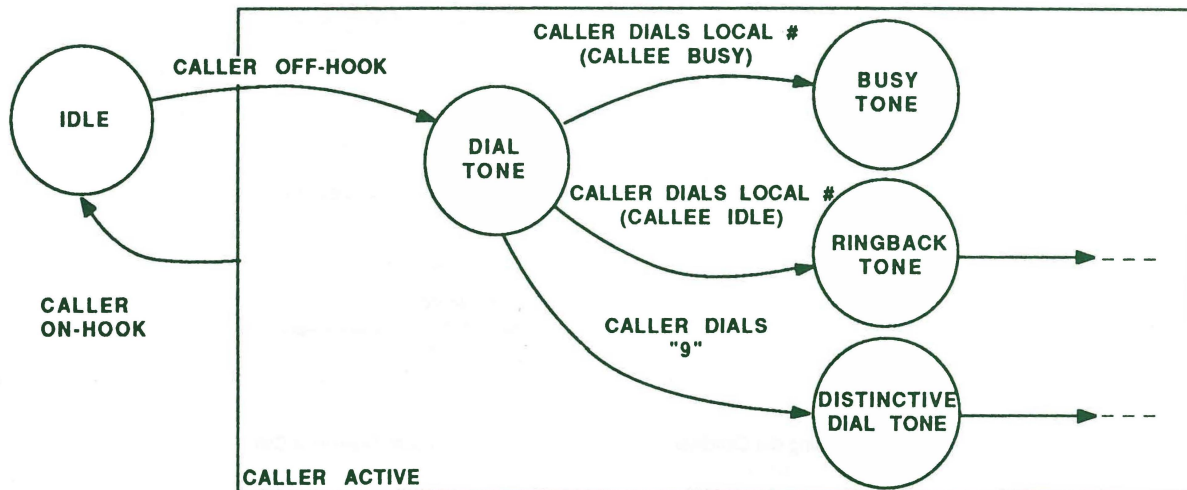


FIGURE 12. Using the Superstate Extension in Telephony

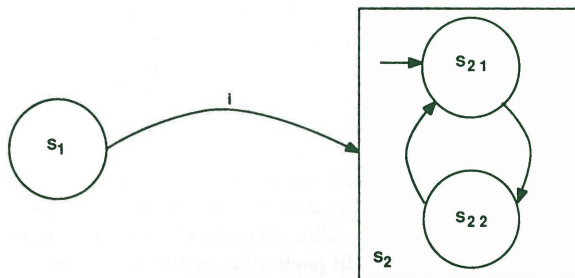


FIGURE 13. Default Entry State Example

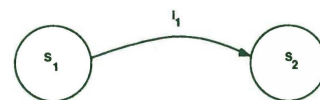
transition from state S_{213} to state S_{211} will only occur if machine (superstate) S_{22} is in state S_{222} .

In summary, Harel's statecharts provide natural extensions to FSMs to make them more suitable for the specification of external behavior of real-time systems. These extensions provide for the hierarchical decomposition of states and the specification of transitions dependent on global conditions and on being in particular states.

Requirements Engineering Validation System

The *Requirements Engineering Validation System* (REVS) [28] is a set of tools that analyzes requirements written in the Requirements Statement Language (RSL) [6] developed using the Software Requirements Engineering Methodology (SREM) [2]. The tools, language, and methodology were developed by TRW, Inc., for the U.S. Army Ballistic Missile Defense (now called the Strategic Defense Command) Advanced Technology Center in Huntsville, Alabama. RSL and its corresponding graphical notation, R-nets [4], represent an extension to conventional FSMs. In particular, for some large, complex, real-time systems, it may not make sense to describe the external behavior of the entire system as one huge finite state machine.

As described earlier, Harel's extensions, which provide the hierarchical decomposition of FSMs, may not be easy or straightforward to apply. In particular, some applications are *stimulus rich*. Stimulus rich implies that customers view the system they want to buy or build in terms of its rich set of stimuli. For such systems, it is easier for customers to think of their requirements for the solution system's external behavior as organized into a set of units, each describing all required system responses to a single stimulus, R-nets. One can think of



(a) The Highest Level Statechart

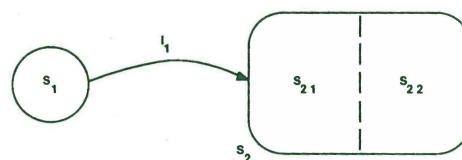
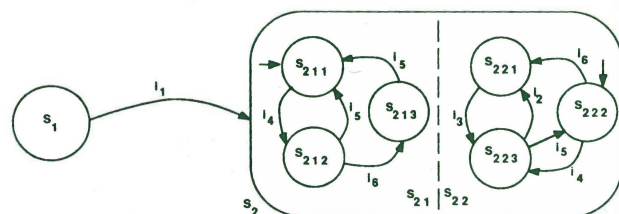
(b) The Refinement of State S_2 (c) The Refinement Of States S_{21} and S_{22}

FIGURE 14. Refinement Using the "And" Function